

DecidArch — Shinobi Arena · versión 2

Resumen de reglas. Juego de decisiones de arquitectura para el proyecto Shinobi Arena (Greenfoot / Java → Unity).

Contenido del mazo

- 1 Project Card · 3 Stakeholder Cards · 12 Concern Cards + 1 Concern condicional (C-15) · 8 Event Cards
- 1 Carta de Glosario de atributos · 1 Hoja de Consistencia entre decisiones
- 1 Decision Preparation Template por jugador · 1 Decision Taking Template por grupo · 1 Registro de Reconsideraciones · 1 Scoring Sheet
- Equipo de 2 a 4 jugadores. Duración: 40 minutos. Hacen falta un lápiz y un reloj.

Preparación

- Dejar a la vista la Project Card, las Stakeholder Cards, el Glosario y la Hoja de Consistencia.
- Formar dos mazos boca abajo y barajados: uno de Concern Cards (12; C-15 queda aparte) y otro de Event Cards.
- Entregar una Decision Preparation Template a cada jugador y una Decision Taking Template al grupo.
- **Las QA-Priority iniciales vienen impresas en cada Stakeholder Card** y se copian al Scoring Sheet antes de empezar. Las Event Cards las modifican durante la partida: pueden subir y también bajar.

Turno (3 minutos)

- El jugador de turno roba una Concern Card y la lee en voz alta.
- **1 minuto:** cada jugador, de forma independiente y en silencio, elige una opción y anota su rationale en su plantilla individual.
- **2 minutos:** el grupo discute, decide una sola opción y la registra en la Decision Taking Template, indicando además qué clase o componente del juego queda afectado.
- Antes de cerrar el turno, revisar la Hoja de Consistencia: si la decisión contradice una anterior, hay que cambiar una de las dos o anotar la penalización.
- Pasa el turno al siguiente jugador.

Event Cards

- Al terminar la ronda, cuando todos han jugado su turno, se roba una Event Card.
- El evento cambia el contexto del proyecto y modifica QA-Priority. Sus efectos duran el resto de la partida.
- Hay que reconsiderar las decisiones ya tomadas a la luz del cambio. **Toda reconsideración se anota en el Registro de Reconsideraciones.**
- Si el evento menciona un concern que todavía no ha salido, se deja la carta del evento a la vista: cuando ese concern se robe, se resuelve ya con el efecto aplicado.
- Si no quedan Event Cards, se sigue jugando sin robar.

Retrabajo

- Cambiar una decisión ya registrada es legítimo y es parte del aprendizaje, pero cuesta: **cada cambio suma 1 punto a la casilla R** del Scoring Sheet (máximo 5).
- Los impactos que se cuentan al final son los de la **opción vigente** al terminar la partida, no los de la opción abandonada.

Opciones propias y símbolo <?>

- Los jugadores pueden proponer opciones nuevas en cualquier momento, anotándolas en la columna de opción.
- También pueden corregir los impactos de una opción existente, justificándolo en la columna Rationale.
- La lista de impactos de cada opción termina en «...» porque no es cerrada. Escriban sobre las plantillas, nunca sobre las cartas.

Fin de la partida. Se juega hasta agotar los 40 minutos o resolver todas las Concern Cards. Después se calcula el resultado con el Scoring Sheet. Un concern sin resolver no cuesta lo mismo que otro: se descuenta según su prioridad.

Project Card

Project:

Shinobi Arena

Purpose:

Shinobi Arena es un videojuego de peleas 2D de uno contra uno, con vista lateral y control por teclado, que se instala y se ejecuta localmente. El jugador compite en el Torneo de los Cinco Reinos enfrentando rivales sucesivos, cada uno con un estilo de combate distinto, hasta el oponente final.

Restricciones:

El prototipo se construye en Greenfoot con Java y debe portarse a Unity en la unidad siguiente. Lo desarrolla un equipo de estudiantes durante un semestre, con entregas parciales cada semana. El prototipo mínimo exige un jugador, dos enemigos con comportamiento distinto, dos arenas, barras de vida y chakra, victoria, derrota, menú y reinicio.

Fuera de alcance:

Multijugador en red, guardado de progreso, sonido y música, y cualquier contenido que no sea original del equipo.

Stakeholder Card

Project:

Shinobi Arena

Stakeholder:

Jugador

Goal:

El Jugador quiere sentarse frente al teclado y tener un combate que se sienta justo: que el personaje responda a lo que ordena, entender por qué perdió cuando pierde, y poder retomar el juego semanas después sin reaprender los controles.

Quality Attributes · QA-Priority inicial:

Satisfaction — Fluidez y respuesta del combate: que el personaje haga lo ordenado, cuando se ordena. **(QA-Priority: 2)**

Reliability — Ausencia de estados rotos, personajes trabados y comportamiento impredecible. **(QA-Priority: 2)**

Learnability — Cuánto tarda un jugador nuevo en entender los controles y las mecánicas básicas. **(QA-Priority: 1)**

Accessibility — Cuánto puede adaptarse el juego a quien lo usa: teclas, ritmo, claridad. **(QA-Priority: 1)**

Performance — Coste de cálculo por cuadro y estabilidad de la velocidad en pantalla. **(QA-Priority: 1)**

Stakeholder Card

Project:

Shinobi Arena

Stakeholder:

Equipo de desarrollo

Goal:

El Equipo construye el prototipo, lo mantiene durante el semestre y debe poder portarlo a Unity más adelante. Necesita depurar rápido y agregar contenido sin reescribir lo que ya funciona.

Quality Attributes · QA-Priority inicial:

Modifiability — Esfuerzo para agregar un personaje, un ataque o una arena sin tocar lo existente. **(QA-Priority: 2)**

Analysability — Facilidad para leer el código y localizar la responsabilidad de cada clase. **(QA-Priority: 2)**

Testability — Poder comprobar una regla de combate sin tener que jugar una partida completa. **(QA-Priority: 1)**

Portability — Esfuerzo para llevar el juego de Greenfoot a Unity y a otros sistemas operativos. **(QA-Priority: 1)**

Stakeholder Card

Project:

Shinobi Arena

Stakeholder:

Revisor técnico (docente)

Goal:

El Revisor abre el proyecto sin conocimiento previo y debe poder explicar qué hace cada clase, verificar que el comportamiento común no esté duplicado y comprobar las reglas del combate de forma aislada.

Quality Attributes · QA-Priority inicial:

Analysability — Facilidad para leer el código y localizar la responsabilidad de cada clase. **(QA-Priority: 2)**

Testability — Poder comprobar una regla de combate sin tener que jugar una partida completa. **(QA-Priority: 1)**

Reliability — Ausencia de estados rotos, personajes trabados y comportamiento impredecible. **(QA-Priority: 1)**

Modifiability — Esfuerzo para agregar un personaje, un ataque o una arena sin tocar lo existente. **(QA-Priority: 1)**

Concern Card

Concern-ID:

Prioridad 5,00

C-02

Concern:

El jugador y el enemigo comparten vida, chakra, gravedad y daño, y solo se diferencian en quién decide sus acciones. ¿Cómo debe repartirse ese comportamiento entre clases?

Options:

- 1 Dos clases independientes: Player y Enemy heredan de Actor por separado, cada una con su propia copia de la lógica de vida, chakra y física.
Performance (+), Modifiability (- -), Analysability (-), Testability (-), Reliability (-), ...
- 2 Clase abstracta Fighter: concentra vida, chakra, física y daño; Player y Enemy la extienden y solo redefinen cómo se decide la acción.
Modifiability (+ +), Analysability (+ +), Reliability (+), Testability (+), ...
- 3 Composición por componentes: Fighter contiene objetos Movimiento, Combate y Salud intercambiables entre sí.
Modifiability (+ +), Testability (+ +), Portability (+), Analysability (- -), Performance (-), ...

<?>

Concern Card

Concern-ID:

Prioridad 4,80

C-01

Concern:

Un luchador puede estar quieto, caminando, saltando, atacando, bloqueando, golpeado o derrotado, y hay combinaciones prohibidas: quien está siendo golpeado no puede atacar. ¿Cómo se representa su estado?

Options:

- 1 Banderas booleanas dentro de Fighter: atacando, saltando y bloqueando, consultadas con if anidados.
Performance (+), Reliability (- -), Modifiability (- -), Analysability (-), Satisfaction (-), ...
- 2 Un enum FighterState con un switch en act() que decide qué acciones se permiten en cada estado.
Reliability (+), Satisfaction (+), Analysability (+), Performance (+), Modifiability (-), ...
- 3 Patrón State: una clase por estado (IdleState, AttackingState, HurtState...), cada una decide sus propias transiciones.
Reliability (+ +), Modifiability (+ +), Analysability (+ +), Satisfaction (+), Testability (+), Performance (-), ...

<?>

Concern Card

Concern-ID:

Prioridad 4,30

C-04

Concern:

Un golpe conecta durante pocos cuadros y solo en una zona por delante del personaje; un proyectil impacta en otro momento y con otra forma. ¿Cómo se detecta que un ataque impacta?

Options:

- 1 Intersección de los sprites completos con getIntersectingObjects mientras dura el ataque.
Performance (+ +), Analysability (+), Satisfaction (- -), Modifiability (-), Testability (-), ...
- 2 Un actor Hitbox temporal que el ataque crea durante sus cuadros activos y destruye al terminar.
Satisfaction (+ +), Modifiability (+ +), Learnability (+), Analysability (+), Performance (-), ...
- 3 Cálculo geométrico de rectángulos dentro del árbitro del combate, sin crear actores adicionales.
Performance (+ +), Testability (+ +), Portability (+), Satisfaction (+), Analysability (- -), ...

<?>

Proyectil reutiliza el mismo mecanismo que se elija aquí.

Concern Card

Concern-ID:

Prioridad 4,20

C-05

Concern:

Alguien debe aplicar el daño, restar chakra, decidir si el bloqueo lo reduce, detectar que la vida llegó a cero y declarar el resultado. ¿Qué componente arbitra las reglas del combate?

Options:

- 1 FightWorld hace todo: crea los actores, aplica el daño y decide el final.
Performance (+), Analysability (- -), Modifiability (- -), Testability (- -), Portability (- -), ...
- 2 Un CombatManager colaborador: FightWorld le delega las reglas; el manager no sabe dibujar ni leer teclado, y no depende de la API de Greenfoot.
Analysability (+ +), Modifiability (+ +), Testability (+ +), Portability (+ +), Reliability (+), ...
- 3 Un CombatManager Singleton accesible desde cualquier clase.
Performance (+), Testability (- -), Reliability (- -), Modifiability (-), Portability (-), ...

<?>

Concern Card

Concern-ID:

Prioridad 3,70

C-09

Concern:

El juego tiene pantalla de inicio, combate, victoria, derrota y reinicio. ¿Cómo se organiza el paso entre esas pantallas?

Options:

1 Un solo World que cambia su fondo y dibuja el menú o el combate según una variable de estado interna.
Performance (+), Analysability (- -), Modifiability (- -), Reliability (-), Learnability (-), ...

2 Tres World separados (MenuWorld, FightWorld, EndWorld) enlazados con Greenfoot.setWorld().
Analysability (+ +), Modifiability (+ +), Reliability (+ +), Learnability (+ +), Testability (+), Portability (-), ...

3 Un World con un objeto GameState que delega el dibujado y el input al estado activo.
Portability (+ +), Modifiability (+), Testability (+), Reliability (+), Analysability (-), Learnability (-), ...

<?>

Concern Card

Concern-ID:

Prioridad 3,60

C-08

Concern:

Kaiken usa técnicas de fuego, Raiko es eléctrico y veloz, Sora ataca a distancia con viento. ¿Cómo se representa la diferencia de estilo y habilidades entre personajes?

Options:

1 Un if por personaje dentro de Fighter.usarEspecial(), consultando el nombre del personaje y sus estadísticas.
Performance (+), Modifiability (- -), Analysability (- -), Accessibility (-), Testability (-), ...

2 Una subclase por personaje que redefine usarEspecial().
Modifiability (+ +), Analysability (+ +), Accessibility (+), Reliability (+), Testability (-), ...

3 Un objeto Tecnica intercambiable que el luchador contiene por composición y puede cambiarse en caliente.
Modifiability (+ +), Testability (+ +), Accessibility (+ +), Portability (+), Analysability (-), Performance (-), ...

<?>

Concern Card

Concern-ID:

Prioridad 3,60

C-03

Concern:

El enemigo debe acercarse, retroceder, atacar a distancia adecuada, bloquear de manera ocasional y usar su especial, sin caer en una secuencia fija. ¿Dónde vive esa lógica de decisión?

Options:

1 Una cadena de if sobre la distancia al jugador, escrita dentro de Enemy.act().
Performance (+), Modifiability (- -), Testability (- -), Satisfaction (-), Accessibility (-), ...

2 Una clase EnemyAI separada a la que Enemy delega: la IA devuelve una acción y el enemigo la ejecuta.
Modifiability (+ +), Testability (+ +), Analysability (+), Portability (+), Reliability (+), ...

3 Un autómata de estados de IA (Perseguir, Atacar, Retroceder, Esperar) con cooldowns y transiciones probabilísticas.
Satisfaction (+ +), Accessibility (+ +), Modifiability (+), Reliability (+), Performance (-), Analysability (-), ...

<?>

Concern Card

Concern-ID:

Prioridad 3,40

C-13

Concern:

El jugador debe entender los controles en segundos y, en la feria, jugar de pie con un teclado ajeno. ¿Dónde vive la lectura del teclado y cómo se cambia el mapeo de teclas?

Options:

1 Cada luchador consulta Greenfoot.isKeyDown() directamente dentro de su act().
Performance (+), Accessibility (- -), Modifiability (- -), Testability (- -), Learnability (-), ...

2 Una clase InputHandler que traduce teclas a acciones (Accion.ATACAR, Accion.BLOQUEAR...) y a la que Player consulta; el mapeo vive en un solo lugar.
Accessibility (+ +), Modifiability (+ +), Testability (+ +), Learnability (+), Portability (+), ...

3 InputHandler más una pantalla de controles en el menú que muestra y permite reasignar las teclas.
Learnability (+ +), Accessibility (+ +), Satisfaction (+), Modifiability (+), Analysability (-), Performance (-), ...

<?>

Carta añadida en la revisión: si en su priorización original C-10 o C-11 tratan este tema, renumeren.

Concern Card

Concern-ID:

Prioridad 3,20

C-14

Concern:

Un jugador nuevo debe entender al instante si su golpe conectó, si se lo bloquearon y cuánto chakra le queda. ¿Cómo se comunica el estado del combate a quien juega?

Options:

- 1 Solo las barras de vida y chakra; el resto se deduce del movimiento de los personajes.
Performance (+), Learnability (- -), Satisfaction (- -), Accessibility (-), ...
- 2 Cada Fighter se encarga de su propio aviso: parpadea al recibir daño y cambia de imagen al bloquear, con ese código dentro del personaje.
Satisfaction (+ +), Learnability (+), Modifiability (- -), Analysability (-), ...
- 3 Una capa HUD que observa los eventos del combate (golpe, bloqueo, KO) y dibuja los avisos; los luchadores no conocen la interfaz.
Learnability (+ +), Satisfaction (+ +), Accessibility (+), Modifiability (+ +), Analysability (+), Performance (-), ...

<?>

Carta añadida en la revisión. Se apoya en lo que decidan en C-06.

Concern Card

Concern-ID:

Prioridad 2,90

C-12

Concern:

El prototipo necesita dos arenas que se diferencian en fondo, altura del suelo, límites laterales y posiciones iniciales. ¿Cómo se configura un escenario de combate?

Options:

- 1 Una subclase de World por arena: BosqueWorld y TemploWorld.
Reliability (+), Performance (+), Modifiability (-), Analysability (-), ...
- 2 FightWorld parametrizado con un objeto Arena que aporta fondo, suelo, límites y puntos de aparición.
Analysability (+ +), Modifiability (+), Testability (+), Portability (+), Reliability (+), ...
- 3 Arenas definidas en un archivo de datos que se lee al iniciar el combate.
Modifiability (+ +), Portability (+), Testability (+), Reliability (- -), Analysability (-), ...

<?>

Concern Card

Concern-ID:

Prioridad 2,80

C-07

Concern:

El prototipo necesita un jugador y dos enemigos distintos, y más adelante posiblemente más personajes. ¿Cómo se crean los luchadores y sus variantes?

Options:

- 1 Con new directo en FightWorld, escribiendo ahí los parámetros de cada personaje.
Performance (+), Modifiability (- -), Analysability (-), Testability (-), ...
- 2 Una FighterFactory con un método que recibe el tipo de personaje y devuelve el Fighter ya configurado.
Modifiability (+ +), Analysability (+ +), Testability (+), Reliability (+), ...
- 3 Estadísticas en un archivo de configuración externo que la fábrica lee al crear cada luchador.
Modifiability (+ +), Testability (+ +), Portability (+), Reliability (-), Analysability (-), ...

<?>

Concern Card

Concern-ID:

Prioridad 2,60

C-06

Concern:

La barra de vida y la de chakra deben reflejar el estado del luchador en todo momento. ¿Cómo se mantienen sincronizadas con él?

Options:

- 1 Polling: FightWorld pregunta getVida() a cada luchador en cada act() y redibuja las barras.
Analysability (+), Modifiability (-), Performance (-), Satisfaction (-), ...
- 2 Observer: el Fighter notifica a sus observadores cuando su vida o chakra cambia, y las barras se actualizan solas.
Modifiability (+ +), Performance (+), Satisfaction (+), Learnability (+), Reliability (+), Analysability (-), ...
- 3 Referencia directa: cada Fighter guarda un puntero a su HealthBar y la actualiza al recibir daño.
Performance (+ +), Satisfaction (+), Modifiability (- -), Portability (-), Testability (-), ...

<?>

Concern Card

Concern-ID:

Prioridad 2,40

C-15

Concern:

El combate ahora se juega al mejor de tres asaltos y con cronómetro: si el tiempo se agota gana quien tenga más vida. ¿Quién controla el tiempo y el fin del asalto?

Options:

-
- 1** Un contador de cuadros dentro de FightWorld.act().
Performance (+), Modifiability (-), Testability (- -), Analysability (-), ...

 - 2** El árbitro del combate lleva el reloj y decide el fin por tiempo junto con el fin por KO.
Analysability (+ +), Testability (+ +), Modifiability (+), Reliability (+), ...

 - 3** Un objeto MatchTimer independiente al que el HUD observa para mostrar el cronómetro.
Modifiability (+ +), Testability (+), Satisfaction (+), Learnability (+), Analysability (-), Performance (-), ...

<?>

CARTA CONDICIONAL: entra al mazo solo si sale el evento «Nuevo requisito: asaltos y cronómetro». La opción 2 exige haber elegido un árbitro separado en C-05.

Event Card

Title:

Playtest: los golpes no se sienten

Description:

Quienes prueban el prototipo reportan que reciben daño sin ver el golpe conectar y que a veces el personaje queda trabado a mitad de un ataque.

Satisfaction del Jugador sube a QA-Priority **3**. Reconsideren **C-04**, **C-01** y **C-14**.

Event Card

Title:

El porteo a Unity se adelanta

Description:

La siguiente unidad del proyecto exige rehacer el juego en Unity. Todo lo que dependa de la API de Greenfoot habrá que reescribirlo desde cero.

Portability del Equipo de desarrollo sube a QA-Priority **3**. Reconsideren **C-05**, **C-09** y **C-04**.

Event Card

Title:

Se incorpora un desarrollador nuevo

Description:

Una persona sin contexto se suma al equipo y debe poder trabajar sobre el código existente sin que se lo expliquen entero.

Analysability del Equipo de desarrollo sube a QA-Priority **3**. Reconsideren **C-02** y la carta que más clases haya agregado al diseño.

Event Card

Title:

Torneo de exhibición en la universidad

Description:

El juego se mostrará en una feria abierta donde la gente juega una sola partida, de pie, con un teclado ajeno y sin explicación previa.

Learnability del Jugador sube a QA-Priority **3** y **Accessibility** a **2**. Reconsideren **C-13**, **C-14** y **C-09**.

Event Card

Title:

Se agregan dos personajes jugables

Description:

El alcance del proyecto crece: hay que sumar dos luchadores más con técnicas propias, reutilizando los sistemas existentes.

Si en **C-08** eligieron la opción 1, la deuda se paga ahora: cuenten **Modifiability (- -)** una segunda vez. Si eligieron 2 o 3, sumen **Modifiability (+)**. Reconsideren **C-08** y **C-07**.

Event Card

Title:

Auditoría de originalidad del contenido

Description:

Se revisa que ningún nombre, técnica ni recurso gráfico provenga de una obra con derechos de autor. Todo lo que no tenga origen documentado debe reemplazarse.

Los datos y recursos de cada personaje deben poder sustituirse sin tocar el resto del sistema. Reconsideren **C-07** y registren en la columna Rationale qué atributo los protege.

Event Card

Title:

Recorte de alcance: se cancela el porteo

Description:

La unidad de Unity se evaluará con un proyecto nuevo y no con el porteo de este juego. Lo que se hizo solo por portabilidad deja de pagar.

Portability del Equipo de desarrollo baja a QA-Priority **0**. Pueden revertir cualquier decisión tomada solo por portabilidad; anoten el motivo en Reconsideraciones.

Event Card

Title:

Nuevo requisito: asaltos y cronómetro

Description:

El profesor pide que el combate se juegue al mejor de tres asaltos y con tiempo límite, como en los juegos de pelea comerciales.

Agreguen la carta condicional **C-15** al mazo de Concerns y resuélvanla antes de terminar. Si queda sin resolver, cuenta en la casilla D del Scoring Sheet.

Glosario de atributos de calidad

Nombres alineados con ISO/IEC 25010. Se usan en inglés porque así aparecen en las cartas y en la bibliografía del ramo.

Atributo	En español	Qué se pregunta el grupo al puntuarlo
Learnability	Facilidad de aprendizaje	Cuánto tarda alguien en usar el sistema por primera vez sin ayuda.
Satisfaction	Satisfacción	Qué tan bien responde el sistema a lo que la persona espera mientras lo usa.
Accessibility	Accesibilidad	Hasta qué punto personas con distintas capacidades y contextos pueden usarlo.
Performance	Eficiencia de desempeño	Coste de cálculo y memoria para el trabajo que hace; en Greenfoot, por cuadro.
Reliability	Fiabilidad	El sistema hace lo que debe sin fallar ni quedar en estados inválidos.
Modifiability	Modificabilidad	Esfuerzo para cambiar o extender el sistema sin romper lo existente.
Analysability	Analizabilidad	Esfuerzo para entender el código y localizar la causa de un fallo.
Testability	Testeabilidad	Facilidad para comprobar una parte de forma aislada.
Portability	Portabilidad	Esfuerzo para llevar el sistema a otro entorno o tecnología.

Hoja de Consistencia entre decisiones

Las decisiones no son independientes: elegir una opción cambia lo que conviene en otra carta. Revisen esta hoja al cerrar cada turno y al final de la partida.

Tipo	Combinación	Efecto
Contradicción	C-05 opción 1 (FightWorld hace todo) con C-02 opción 3, C-06 opción 2 o C-15 opción 2.	Si el World arbitra todo, no hay a quién delegar. Cambien una de las dos decisiones o cuenten Analysability (-) y Testability (-) extra.
Contradicción	C-01 opción 1 (banderas booleanas) con C-04 opción 2 (Hitbox temporal).	Las banderas no saben cuándo termina el ataque y el Hitbox queda huérfano: cuenten Reliability (-) extra.
Contradicción	C-08 opción 1 (if por personaje) con el evento «Se agregan dos personajes jugables».	Ya está previsto en la carta de evento: la deuda se paga ahí.
Sinergia	C-07 opción 2 (Factory) con C-08 opción 3 (Técnica por composición).	La fábrica arma el luchador y le inyecta su técnica: sumen Modifiability (+) .
Sinergia	C-06 opción 2 (Observer) con C-14 opción 3 (capa HUD que observa).	Es la misma infraestructura de notificación para barras y avisos: sumen Analysability (+) .
Sinergia	C-05 opción 2 (CombatManager) con C-03 opción 2 (EnemyAI separada).	Reglas y decisiones quedan fuera de Greenfoot y se pueden probar sin jugar: sumen Testability (+) .
Cuidado	C-12 opción 3 y C-07 opción 3 juntas (dos archivos de datos externos).	Definan un único formato y una única clase que lo lea, o cuenten Reliability (-) extra.

Regla general. Si dos decisiones se contradicen, el grupo debe cambiar una (cuesta 1 punto de retrabajo) o justificar en Rationale por qué conviven. Una contradicción no resuelta y no justificada cuesta 2 puntos en la casilla R del Scoring Sheet.
--

Decision Preparation Template (Individual)

Cada jugador recibe una plantilla individual. Se llena en silencio, antes de discutir con el grupo.

Group-ID: _____ Jugador: _____

[illegible]

Decision Taking Template (Group)

Registra las decisiones del grupo. La columna de clase o componente convierte la partida en material directo para el diagrama de clases y el C4.

Group-ID:

[illegible]

Registro de Reconsideraciones

Cada vez que un evento obliga a cambiar una decisión ya tomada, anótenlo aquí. Es la evidencia de que el diseño evolucionó y de por qué: sirve para el informe y suma a la casilla R.

Ronda	Event Card	Concern-ID	Decisión anterior	Decisión nueva	Motivo del cambio

R = número de decisiones cambiadas (máximo 5) **+ 2 por cada contradicción no resuelta.** Anoten el total en el Scoring Sheet: R = _____

Scoring Sheet

Se completa al terminar la partida.

Paso 1 · QA-Scores

Quality Attribute	Learnability	Satisfaction	Accessibility	Performance	Reliability	Modifiability	Analysability	Testability	Portability
A = suma de "+" ("+" + "vale 2)									
B = suma de "-" ("- -" vale 2)									
QA-Score = A - B									

Se cuentan solo los impactos de las opciones vigentes al terminar la partida, incluidos los que el equipo haya agregado y los ajustes de la Hoja de Consistencia y de las Event Cards.

Salud técnica: anoten cuántos atributos terminaron con QA-Score negativo: _____. Si son **3 o más**, el diseño no es viable y el equipo pierde la partida, sin importar el puntaje.

Paso 2 · Stakeholders-Score

Stakeholder	Quality Attribute	QA-Score	QA-Priority inicial	QA-Priority final	C = QA-Score - QA-Priority final
Jugador	Satisfaction		2		
	Reliability		2		
	Learnability		1		
	Accessibility		1		
	Performance		1		
Equipo de desarrollo	Modifiability		2		
	Analysability		2		
	Testability		1		
	Portability		1		
Revisor técnico (docente)	Analysability		2		
	Testability		1		
	Reliability		1		
	Modifiability		1		
Suma de C por stakeholder — Jugador: _____ Equipo: _____ Revisor: _____					
Stakeholders-Score = suma de todos los valores de C					

Usar la QA-Priority vigente al terminar la partida, ya modificada por las Event Cards. Un atributo que aparece en dos stakeholders cuenta dos veces: es intencional, refleja que más de un interesado depende de él.

Satisfacción: un stakeholder queda insatisfecho si la suma de sus valores de C es negativa. Si dos de los tres quedan insatisfechos, el equipo pierde la partida.

Paso 3 · Final Score

D = suma de las prioridades de las Concern Cards que quedaron sin resolver, redondeada	
R = retrabajo (Registro de Reconsideraciones)	
Final Score = Stakeholders-Score - D - R	

Final Score	Resultado
Menor que 0, o alguna condición de derrota de los pasos 1 o 2	Perdieron la partida
0 a 29	Suficiente
30 a 59	Bueno
60 a 89	Muy bueno
90 o más	Excelente

Las bandas son orientativas: sirven para comparar grupos que jugaron el mismo mazo. Lo que evalúa el ramo es el rationale escrito, no el número.

Hoja del facilitador

Para quien dirige la sesión: tiempos, cierre y cómo convertir la partida en el entregable de la semana.

Tiempos sugeridos (55 minutos en total)

Minutos	Actividad
0 - 5	Leer la Project Card y las tres Stakeholder Cards en voz alta. Copiar las QA-Priority iniciales al Scoring Sheet.
5 - 45	Partida: turnos de 3 minutos, Event Card al cerrar cada ronda. El facilitador avisa el tiempo y no opina sobre las decisiones.
45 - 52	Scoring en grupo, en voz alta, discutiendo cada impacto discutible.
52 - 55	Reflexión con las preguntas de abajo.

Preguntas de cierre

- ¿Qué decisión cambió el grupo por culpa de un evento y no por un argumento técnico? ¿Habría sido evitable decidiendo distinto la primera vez?
- ¿Alguna opción se eligió porque sonaba más profesional y no porque el proyecto la necesitara? Los patrones cuestan Analysability: ¿se pagó ese coste a cambio de algo?
- ¿Qué atributo quedó sacrificado en todas las rondas? ¿Fue una decisión del grupo o pasó sin que nadie lo notara?
- ¿En qué se diferencia lo que cada uno propuso en solitario de lo que el grupo terminó eligiendo?

Qué se entrega después de jugar

- **Decision Taking Template completa:** es la lista de decisiones de arquitectura del proyecto, con su justificación. Cada fila es una decisión defendible frente al profesor.
- **Columna de clase o componente:** alimenta directamente el diagrama de clases y la vista de componentes del C4.
- **Registro de Reconsideraciones:** evidencia de que el diseño evolucionó; es lo que pide la vista dinámica del C4 y la estrategia de refactoring.
- **Scoring Sheet:** muestra qué atributos se priorizaron y cuáles se sacrificaron, con números.

Advertencia sobre el mazo. Ninguna columna de opciones es «la correcta»: la opción 3 suele ser la más elaborada y casi siempre cobra en Analysability o en Performance. Un grupo que elige siempre la 3 no gana automáticamente. Si el grupo detecta que un impacto está mal calibrado, que lo corrija y lo justifique: eso es exactamente lo que el juego quiere entrenar.